Baze de date II

CURS:
FLORIN RADULESCU, CIPRIAN TRUICA

Site:
cs.curs.pub.ro

Mod de notare

1. 60% in timpul semestrului:
   - Prezenta la curs – 10% (10p)
   - Laborator – 35% (6p)
   - Tema tip proiect – 15%

2. 40% Verificarea finala (examen)

Baza de date utilizata. Tabela STUD

MATR	NUME	AN	GRUPA	DATAN	LOC	TUTOR	PUNCTAJ	CODS
1456	GEORGE	4	1141A	12-MAR-82	BUCURESTI		2890	11
1325	VASILE	2	1122A	05-OCT-84	PITESTI	1456	390	11
1645	MARIA	3	1131B	17-JUN-83	PLOIESTI		1400	11
3145	ION	1	2112B	24-JAN-85	PLOIESTI	3251	1670	21
2146	STANCA	4	2141A	15-MAY-82	BUCURESTI		620	21
3251	ALEX	5	2153B	07-NOV-81	BRASOV		1570	21
2215	ELENA	2	2122A	29-AUG-84	BUCURESTI	2146	890	21
4311	ADRIAN	3	2431A	31-JUL-83	BUCURESTI		450	24
3514	FLOREA	5	2452B	03-FEB-81	BRASOV		3230	24
1925	OANA	2	2421A	20-DEC-84	BUCURESTI	4311	760	24
2101	MARIUS	1	2412B	02-SEP-85	PITESTI	3514	310	24
4705	VOICU	2	2421B	19-APR-84	BRASOV	4311	1290	24

Tabelele SPEC si BURSA

CODS	NUME	DOMENIU
11	MATEMATICA	STIINTE EXACTE
21	GEOGRAFIE	UMANIST
24	ISTORIE	UMANIST

TIP	PMIN	PMAX	SUMA
FARA BURSA	0	399	
BURSA SOCIALA	400	899	100
BURSA DE STUDIU	900	1799	150
BURSA DE MERIT	1800	2499	200
BURSA DE EXCEPTIE	2500	9999	300

Functii

- Expresiile SQL şi PL/SQL pot conţine funcţii din tabelul următor.

Există însă restricţiile următoare:

- Funcţiile SQLCODE şi SQLERRM nu pot fi folosite în cereri SQL.

Functii

Există însă restricţiile următoare:

- Funcţiile DEREF, REF, VALUE, DECODE, DUMP, GREATEST, LEAST şi VSIZE nu pot fi folosite în instrucţiunile procedurale ale PL/SQL (în documentaţia Oracle9 nu mai sunt amintite în această categorie GREATEST şi LEAST).
- Funcţiile statistice şi analitice (ex: AVG, SUM, COUNT, MIN, MAX, etc.) sunt specifice SQL şi nu pot fi folosite în instrucţiunile procedurale ale PL/SQL.

Functii

Erori	Număr	Caracter	Conversie
SQLCODE	ABS	ASCII	CHARTOROWID
SQLERRM	ACOS	CHR	CONVERT
	ASIN	CONCAT	HEXTORAW
	ATAN	INITCAP	RAWTOHEX
	ATAN2	INSTR	ROWIDTOCHAR
	BITAND	INSTRB	TO_BLOB
	CEIL	LENGTH	TO_CHAR
	COS	LENGTHB	TO_CLOB
	COSH	LOWER	TO_DATE
	EXP	LPAD	TO_MULTI_BYTE
	FLOOR	LTRIM	TO_NCLOB
	LN	NLS_INITCAP	TO_NUMBER
	LOG	NLS_LOWER	TO_SINGLE_BYTE
	MOD	NLSSORT	
	POWER	NLS_UPPER	
	ROUND	REPLACE	
	SIGN	RPAD	
	SIN	RTRIM	
	SINH	SOUNDEX	
	SQRT	SUBSTR	
	TAN	SUBSTRB	
	TANH	TRANSLATE	
	TRUNC	TRIM	
		UPPER	

Functii

Dată	Obiect	Diverse
ADD_MONTHS	DEREF	BFILENAME
CURRENT_DATE	REF	DECODE
CURRENT_TIMESTAMP	VALUE	DUMP
DBTIMEZONE	TREAT	EMPTY_BLOB
EXTRACT		EMPTY_CLOB
FROM_TZ		GREATEST
LAST_DAY		LEAST
LOCALTIMESTAMP		NLS_CHARSET_DECL_LEN
MONTHS_BETWEEN		NLS_CHARSET_ID
NEW_TIME		NLS_CHARSET_NAME
NEXT_DAY		NVL
NUMTODSINTERVAL		SYS_CONTEXT
NUMTOYMINTERVAL		SYS_GUID
ROUND		UID
SESSIONTIMEZONE		USER
SYSDATE		USERENV

Decizia

- Sintaxa este:

```
IF conditie1 THEN
   instructiuni1;
[ELSIF conditie2 THEN
   instructiuni2;]
. . . . . .
[ELSE
   instructiuni_else;]
END IF;
```

Decizia

- Se execută secvenţa de instrucţiuni corespunzătoare primei condiţii îndeplinite.
- Dacă nici o condiţie nu este adevărată şi există ELSE se execută instrucţiunile de pe această ramură.

Decizia

- Observaţie: ELSIF se scrie legat pe când END IF în două cuvinte.
- În aceste condiţii se pot folosi toţi operatorii de comparaţie, operatorii specifici SQL (BETWEEN, IN, LIKE, IS [NOT] NULL), conectorii logici AND, OR, NOT, paranteze şi orice expresii aritmetice sau de altă natură valide.

Exemplu

```
DECLARE
  v_loc varchar2(10) := 'BUCURESTI'; v_nr   number(2);
BEGIN
  SELECT cods
  INTO v_nr
  FROM stud
  WHERE loc=v_loc;
  dbms_output.put_line('Numarul specializarii este: '
||v_nr);
  IF (v_nr=21) then
      dbms_output.put_line('Este 21');
  ELSIF v_nr < 21 then
      dbms_output.put_line('Mai mic decat 21');
  ELSE
      dbms_output.put_line('Mai mare decat 21');
  END IF;
```

Exemplu

```
EXCEPTION
   WHEN no_data_found THEN
     dbms_output.put_line('Nu exista');
   WHEN too_many_rows THEN
     dbms_output.put_line('Sunt mai
multe');
   WHEN others THEN
     dbms_output.put_line('Eroare
nespecificata');
END;
```

- Rezultat:

```
Sunt mai multe
```

Ciclare: LOOP

- Sintaxa este:

```
LOOP
   Instructiuni;
         EXIT [WHEN conditie];
         -- iesire din ciclu
   Instructiuni;
END LOOP;
```

- Se execută repetat instrucţiunile dintre LOOP şi END LOOP.
- Ieşirea din ciclu se face prin EXIT (condiţionat sau nu).

Ciclare: LOOP

- Exemplu: Blocul următor conţine un ciclu LOOP cu o variabilă contor care parcurge valorile 6, 11, 16, . . . şi al cărui corp conţine un subbloc.
- Subblocul testează pentru numerele respective dacă există o specializare cu acel cod şi dacă da afişează datele despre ea în zona executabilă.
- Dacă nu există o astfel de specializare se afişează un mesaj corespunzător în zona de excepţii a subblocului.

Ciclare: LOOP

```
DECLARE
  v_contor number(2) :=6;
  v_cods number;
  v_nume varchar2(10);
  v_dom varchar2(20);
BEGIN
LOOP
  BEGIN
  SELECT cods, nume, domeniu
  INTO v_cods, v_nume, v_dom
  FROM spec
  WHERE cods=v_contor;
  dbms_output.put_line('Cod '|| v_contor||
                       '  nume '||v_nume||
                       ' domeniu '||v_dom);
```

Ciclare: LOOP

```
EXCEPTION
    WHEN no_data_found THEN
dbms_output.put_line('Nu exista codul '||
    v_contor);
END; -- subbloc
v_contor := v_contor + 5;
EXIT when v_contor > 40;
END LOOP;
EXCEPTION
WHEN others THEN
  dbms_output.put_line('Exceptie');
END;
```

Ciclare: LOOP

- Rezultat:

```
Nu exista codul 6
Cod 11  nume MATEMATICA domeniu STIINTE EXACTE
Nu exista codul 16
Cod 21  nume GEOGRAFIE domeniu UMANIST
Nu exista codul 26
Nu exista codul 31
Nu exista codul 36
```

Ciclare: FOR

- Sintaxa este:

```
FOR contor IN [REVERSE]
val_initiala..val_finala LOOP
Instructiuni;
END LOOP;
```

Ciclare: FOR

- Se execută repetat instrucţiunile dintre FOR şi END LOOP pentru fiecare valoare a contorului.
- Acesta parcurge plaja dintre o valoare iniţială şi o valoare finală (inclusiv pe acestea). Numărul de paşi este |val_finala - val_initiala| + 1.
- Ieşirea din ciclu se face după atingerea valorii finale sau prin EXIT (pentru ieşire forţată).
- Implicit valoarea iniţială este mai mică decât valoarea finală.
- REVERSE specifică o parcurgere descrescătoare de la o valoare iniţială mai mare către o valoare finală mai mică.

Ciclare: FOR

- Exemplu: Asemănător cu exemplul anterior dar se testează toate numerele între 20 şi 25 inclusiv.

```
DECLARE
  v_contor number(2);
  v_cods number;
  v_nume varchar2(10);
  v_dom varchar2(20);
BEGIN
FOR v_contor IN 20..25 LOOP
  BEGIN
  SELECT cods, nume, domeniu
  INTO v_cods, v_nume, v_dom
  FROM spec WHERE cods=v_contor;
  dbms_output.put_line('Cod '|| v_contor||
  ' nume '||v_nume||' domeniu '||v_dom);
```

Ciclare: FOR

```
EXCEPTION
  WHEN no_data_found THEN
    dbms_output.put_line('Nu exista codul '||
  v_contor);
END; -- subbloc
END LOOP;
END;
```

- Rezultat:
- **Nu exista codul 20**
- **Cod 21 nume GEOGRAFIE domeniu UMANIST**
- **Nu exista codul 22**
- **Nu exista codul 23**
- **Cod 24 nume ISTORIE domeniu UMANIST**
- **Nu exista codul 25**

Ciclare: WHILE

- Sintaxa este:

```
WHILE conditie LOOP
Instructiuni;
END LOOP;
```

- Cât timp condiţia este adevărată se execută instrucţiunile dintre WHILE şi END LOOP.
- Se poate folosi EXIT pentru ieşirea forţată din ciclu.

Ciclare: WHILE

- Exemplu: Rescriere cu WHILE pentru exemplul de la LOOP

```
DECLARE
   v_contor number(2) :=6;
   v_cods number;
   v_nume varchar2(10);
   v_dom varchar2(20);
BEGIN
WHILE v_contor <=40 LOOP
   BEGIN
   SELECT cods, nume, domeniu
   INTO v_cods, v_nume, v_dom
   FROM spec
   WHERE cods=v_contor;
   dbms_output.put_line('Cod '|| v_contor||
    ' nume '||v_nume||' domeniu '||v_dom);
```

Ciclare: WHILE

```
EXCEPTION
  WHEN no_data_found THEN
    dbms_output.put_line('Nu exista
codul '|| v_contor);
END; -- subbloc
v_contor := v_contor + 5;
END LOOP;
EXCEPTION
WHEN others THEN
  dbms_output.put_line('Exceptie');
END;
```

Ciclare: WHILE

- Rezultatul este acelaşi cu cel obţinut anterior:

```
Nu exista codul 6
Cod 11 nume MATEMATICA domeniu STIINTE EXACTE
Nu exista codul 16
Cod 21 nume GEOGRAFIE domeniu UMANIST
Nu exista codul 26
Nu exista codul 31
Nu exista codul 36
```

Etichete

- Forma unei etichete este:

<<eticheta>>

- Aceasta se plasează înaintea instrucţiunii.
- Dacă instrucţiunea este un ciclu, în cazul în care avem cicluri imbricate se poate comanda ieşirea din ciclu pe mai multe nivele specificând la EXIT eticheta ciclului părăsit.
- În acest caz la END LOOP se poate adauga opţional eticheta ciclului.

Etichete

- Exemplu:

```
<<ciclu_mare>>
LOOP
- - -
<<ciclu_mic>>
     LOOP
- - -
EXIT ciclu_mic WHEN conditie1;
- - -
EXIT ciclu_mare WHEN conditie2;
END LOOP ciclu_mic;
END LOOP ciclu_mare;
```

Instructiunea CASE

- Sintaxa este:

```
CASE expresie
WHEN 'val1' THEN
  Instructiuni1;
WHEN 'val2' THEN
  Instructiuni2;
- - - - - - - - -
[ELSE
  Instructiuni_else;]
END CASE;
```

Instructiunea CASE

- Se evaluează expresia şi se execută instrucţiunile corespunzătoare valorii acesteia.
- În cazul în care expresia nu este egală cu nici una dintre valori şi există ELSE se execută instrucţiunile asociate acestei ramuri.
- Exemplu: Afişarea domeniului asociat specializării cu codul 21.

Instructiunea CASE

```
DECLARE
  v_dom varchar2(20);
BEGIN
  SELECT domeniu
  INTO v_dom
  FROM spec
  WHERE cods=21;
  CASE v_dom
    WHEN 'UMANIST' THEN
      dbms_output.put_line('DOMENIU UMANIST');
    WHEN 'STIINTE EXACTE' THEN
      dbms_output.put_line('DOMENIU STIINTE EXACTE');
    ELSE
      dbms_output.put_line('ALT DOMENIU');
  END CASE;
END;
Rezultat:
DOMENIU UMANIST
```

Instructiunea CASE

- În cazul absenţei lui ELSE se adaugă implicit ridicarea unei excepţii predefinite:

```
ELSE RAISE CASE_NOT_FOUND;
```

- Exemplu: domeniul pentru specializarea cu cod 21 (UMANIST) nu este printre etichetele CASE şi nu există ELSE. In consecinta va semnala excepţia:
- ORA-06592: CASE not found while executing CASE statement.

- Notă: instrucţiunea CASE apare în Oracle începând cu versiunea 9.

Instructiunea CASE

```
DECLARE
  v_dom varchar2(20);
BEGIN
  SELECT domeniu
  INTO v_dom
  FROM spec
  WHERE cods=21;
  CASE v_dom
    WHEN 'MATEMATICA' THEN
      dbms_output.put_line('DOMENIU UMANIST');
    WHEN 'STIINTE EXACTE' THEN
      dbms_output.put_line('EXACTE');
  END CASE;
END;
```

Exceptii

- În cazul sistemelor de gestiune a bazelor de date erorile (dar nu numai) sunt numite şi ***excepţii***. Un bloc PL/SQL poate conţine între EXCEPTION şi END instrucţiuni care să trateze :
  - erorile returnate de serverul Oracle sau uneltele Oracle
  - situaţii definite de utilizator (excepţii definite de utilizator).

Exceptii

- Excepţiile pot avea asociat un nume (identificator). Unele dintre aceste nume sunt predefinite (pentru o parte dintre erorile returnate de server sau unelte).
- Apariţia unei excepţii este numită şi ***ridicarea unei excepţii*** (exception raising). În acest caz execuţia normală a blocului este întreruptă şi se face ***tratarea excepţiei***: se sare în zona EXCEPTION pentru execuţia instrucţiunilor asociate acesteia.

Exceptii

- Caracteristicile principale ale tratării excepţiilor Oracle sunt următoarele:

1. Apariţia unei excepţii, tratată sau nu, termină execuţia blocului.

- Toate instrucţiunile PL/SQL sau cererile SQL care apar după cererea sau instrucţiunea care a ridicat excepţia nu mai sunt executate.

Exceptii

- Caracteristicile principale ale tratării excepţiilor Oracle sunt următoarele:

2. În cazul în care o excepţie are asociat un tratament, la ridicarea ei se trece la execuţia instrucţiunilor de tratare ***după care se iese din blocul curent fără eroare.***

Exceptii

- Caracteristicile principale ale tratării excepţiilor Oracle sunt următoarele:

3. În cazul în care apare o excepţie care nu este tratată explicit în porţiunea dintre EXCEPTION şi END, ***blocul se termină cu eroare***, excepţia putând fi tratată de blocul înconjurător (dacă există).

Exceptii

- Caracteristicile principale ale tratării excepţiilor Oracle sunt următoarele:

4. În cazul excepţiilor definite de utilizator acestea trebuie ***ridicate explicit*** în zona dintre BEGIN şi EXCEPTION prin instrucţiunea PL/SQL RAISE.

Exemplu

- Un exemplu de situaţie în care serverul returnează o eroare este următorul:

```
DECLARE
v_nume   varchar2(10);
BEGIN
SELECT nume INTO v_nume FROM stud
WHERE cods>20;
END;
```

Exemplu

- În cazul în care cererea SQL returnează mai mult de o singură înregistrare, valoarile rezultate nu pot fi stocate într-o singură variabilă scalară.
- O altă eroare pentru blocul de mai sus este şi nereturnarea nici unei valori de către cererea SQL (în cazul în care nici o specializare nu are codul mai mare ca 20).

Tratarea exceptiilor

- Sintaxa tratării excepţiilor este următoarea:

```
DECLARE
. . . .
BEGIN
. . . .
EXCEPTION
WHEN exceptie_11 OR exceptie_12 . . .THEN
      instructiuni_1
[WHEN exceptie_21 OR exceptie_22 . . . THEN
      instructiuni_2]
.  .  .  .  .  .
[WHEN OTHERS THEN
      Instructiuni_others]
END;
```

Tratarea exceptiilor

- **exceptie_xy** este identificatorul unei excepţii (predefinit sau definit de utilizator). OR se foloseşte când mai multe excepţii au aceleaşi instrucţiuni de tratare.
- **instructiuni_x** reprezintă secvenţa de instrucţiuni PL/SQL sau cereri SQL care se execută la apariţia excepţiilor exceptie_xy.
- **instructiuni_others** reprezintă secvenţa de instrucţtiuni PL/SQL sau cereri SQL care se execută la apariţia unei excepţii netratate de clauzele WHEN anterioare.

Tratarea exceptiilor

- De reţinut:
- WHEN OTHERS, dacă există, este întotdeauna ultima.
- După tratatea excepţiei apărute se iese din blocul curent. În consecinţă este inutilă prezenţa aceleiaşi excepţii pe mai mult de un WHEN.

Tipuri de exceptii

- Excepţiile sunt de trei feluri:

1. Excepţii Oracle predefinite.
2. Excepţii Oracle care nu sunt predefinite
3. Excepţii definite de utilizator

Excepţii Oracle predefinite

- Excepţiile predefinite au deja un identificator asociat.
- Ele sunt ridicate automat la apariţie de către serverul Oracle sau uneltele Oracle.

- Unele dintre excepţiile predefinite ridicate de serverul Oracle (* = incepand cu Oracle9)

Excepţii Oracle predefinite

Numele predefinit al excepţiei	Când este ridicată automat de serverul Oracle
ACCESS_INTO_NULL	Asignarea unei valori unui atribut al unui obiect neiniţializat
CASE_NOT_FOUND (*)	Instrucţiune CASE fără ELSE pentru care nici una dintre etichete nu este egală cu valoarea căutată
COLLECTION_IS_NULL	Aplicarea unei alte metode decat EXISTS unui TABLE sau VARRAY neiniţializat
CURSOR_ALLREADY_OPEN	Încercare de deschidere a unui cursor deja deschis
DUP_VAL_ON_INDEX	Inserarea unei valori duplicat pe o coloană unde acest lucru nu este permis (PRIMARY KEY, UNIQUE, etc)
INVALID_CURSOR	Operaţie cursor invalidă
INVALID_NUMBER	Eroare conversie de la şir de caractere la număr

Excepţii Oracle predefinite

Numele predefinit al excepţiei	Când este ridicată automat de serverul Oracle
LOGIN_DENIED	Autentificare Oracle cu un username/parolă invalide
NO_DATA_FOUND	Cererea SELECT care ar fi trebuit să întoarca o singură linie nu întoarce nici o linie
NOT_LOGGED_ON	Programul PL/SQL execută un acces la server fără să fie realizată în prealabil conexiunea cu acesta
PROGRAM_ERROR	Eroare internă PL/SQL
ROWTYPE_MISMATCH	Diferenţa de tip între două cursoare (unul gazdă, altul PL/SQL) implicate într-o asignare
SELF_IS_NULL (*)	Programul apelează o metodă MEMBER pe o instantă nulă. În acest caz, parametrul implicit SELF - primul transmis unei metode MEMBER - e nul
STORAGE_ERROR	Depăşire memorie pentru PL/SQL sau memoria este coruptă

Excepţii Oracle predefinite

Numele predefinit al excepţiei	Când este ridicată automat de serverul Oracle
SUBSCRIPT_BEYOND_COUNT	Referinţa la o tabelă imbricată sau un obiect de tip VARRAY printr-un index mai mare decât numărul de elemente al colecţiei
SUBSCRIPT_BEYOND_LIMIT	Referinţa la o tabelă imbricată sau un obiect de tip VARRAY printr-un index ilegal (de exemplu -1)
SYS_INVALID_ROWID (*)	Conversia unui şir de caractere la tipul UROWID a întors eroare deoarece şirul nu este un UROWID valid.
TIMEOUT_ON_RESOURCE	Time-out la încercarea de accesare a unei resurse de către Oracle
TOO_MANY_ROWS	Cererea SELECT care ar fi trebuit să întoarcă o singură linie întoarce mai multe linii
VALUE_ERROR	Erori aritmetice, de conversie, de trunchiere sau de violare a unei constrângeri privind o valoare
ZERO_DIVIDE	Împărţire la 0

Exemplu

- Blocul următor va tipări mesajul 'Mai multe linii'.

```
DECLARE
v_nume varchar2(10);
BEGIN
SELECT nume INTO v_nume
FROM stud WHERE cods>20;
EXCEPTION
       WHEN NO_DATA_FOUND THEN
              DBMS_OUTPUT.PUT_LINE('Nu exista date');
       WHEN TOO_MANY_ROWS THEN
              DBMS_OUTPUT.PUT_LINE('Mai multe linii');
       WHEN OTHERS THEN
              DBMS_OUTPUT.PUT_LINE('Alta exceptie');
END;
```

De retinut!

Excepţii predefinite:

1. NU se declară în zona DECLARE
2. NU trebuiesc ridicate explicit prin RAISE
3. Se tratează în zona EXCEPTION

Excepţii Oracle non-predefinite

- Reprezintă alte erori returnate de Oracle, în afara celor predefinite. Aceste erori se pot trata în două moduri:

1. Prin folosirea lui WHEN OTHERS
2. Prin declararea excepţiei (i se dă un nume) şi asocierea unui cod de eroare Oracle în zona DECLARE şi tratarea ei în zona EXCEPTION.

Excepţii Oracle non-predefinite

- Declarare exceptie:

```
DECLARE
nume_exceptie    EXCEPTION;
PRAGMA EXCEPTION_INIT(nume_exceptie,
cod_eroare_Oracle);
BEGIN
.   .   .   .
EXCEPTION
      WHEN nume_exceptie THEN
            instructiuni;
            .   .   .   .
END;
```

- Exemplu: Tratarea erorii cu cod -979 (în SELECT există coloana NUME care nu apare în GROUP BY):

Exemplu

```
DECLARE
v_nume VARCHAR2(10);
v_numar NUMBER;
eroare_grupare    EXCEPTION;
PRAGMA EXCEPTION_INIT(eroare_grupare, -979);
BEGIN
      SELECT NUME, COUNT(*) INTO v_nume, v_numar
   FROM STUD GROUP BY CODS;
EXCEPTION
         WHEN eroare_grupare THEN
            DBMS_OUTPUT.PUT_LINE('Cerere
gresita');
         WHEN OTHERS THEN
            DBMS_OUTPUT.PUT_LINE('A aparut o alta
exceptie');
END;
```

De retinut!

Excepţii Oracle non-predefinite

1. Se declară în zona DECLARE şi se asociază un cod de eroare
2. NU trebuiesc ridicate explicit prin RAISE
3. Se tratează în zona EXCEPTION

Excepţii definite de utilizator

- Un program PL/SQL poate conţine excepţii definite de utilizator. În cazul când programul detectează o situaţie anormală el poate trece în zona de excepţii prin instrucţiunea PL/SQL

```
RAISE nume_exceptie
```

- Astfel de excepţii trebuiesc:
  - Declarate explicit
  - Ridicate explicit

Exemplu

```
DECLARE
prea_putini   EXCEPTION;
v_nrstud      NUMBER;
BEGIN
SELECT count(*) INTO v_nrstud FROM stud WHERE cods=21;
IF (v_nrstud < 5) THEN
       RAISE prea_putini;
END IF;
EXCEPTION
       WHEN prea_putini THEN
        DBMS_OUTPUT.PUT_LINE('Sunt sub 5 studenti');
       WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Alta exceptie');
END;
```

De retinut!

Excepţii definite de utilizator

1. Se declară în zona DECLARE (ca nume)
2. Se ridică explicit cu RAISE
3. Se tratează în zona EXCEPTION

Identificarea unei excepţii

- În cazul în care dorim ca în zona EXCEPTION .. WHEN OTHERS să identificăm ce eroare a apărut putem folosi două funcţii care întorc codul numeric respectiv mesajul text al erorii respective.
- În funcţie de valoarea lor putem să tratăm diferenţiat diferite erori netratate anterior în blocul respectiv.

Identificarea unei excepţii

- Cele două funcţii sunt:

1. SQLCODE - întoarce codul numeric al excepţiei (erorii returnate)
2. SQLERRM - întoarce mesajul text asociat cu acea excepţie

Identificarea unei excepţii

- Codul de eroare returnat poate fi:

Valoare SQLCODE	Descriere
0	Nu a apărut nici o excepţie
1	A apărut o excepţie definită de utilizator
100	A apărut excepţia NO_DATA_FOUND
Număr negativ	Cod eroare returnat de serverul Oracle

Exemplu

```
DECLARE
v_cod NUMBER;
v_text       VARCHAR2(255);
BEGIN
.   .   .   .   .
EXCEPTION
          WHEN OTHERS THEN
               ROLLBACK;
               v_cod := SQLCODE;
               v_text := SQLERRM;
               INSERT INTO ERORI VALUES(v_cod,
v_text);
END;
```

Sfârşitul cursului 2